計算物理学B

第四回付録：array と ndarray

野垣 康介、藤本 悠輝

３種類の配列：名前と役割

名前	できること・特徴
`list`	Python 組込み。異なる型を混ぜられ、要素を追加・削除できます。
`array.array`	標準ライブラリの可変１次元配列。型コードで要素型をそろえます。
`numpy.ndarray`	NumPy の１次元・多次元配列。dtype と shape を持ち、要素ごとの計算や行列計算に便利です。

`np.array(...)` は作成関数、`ndarray` は配列の型です。１次元の結果も `ndarray` です。

作り方と型の指定

```
import array as ar
import numpy as np
a_std = ar.array("d", [1.0, 2.0])
a_np = np.array([1.0, 2.0],
   dtype=float)
print(a_std.typecode, a_np.dtype,
   a_np.shape)
```

出力

```
d float64 (2,)
```

'd' は C の double、'i' は C の int（幅は環境依存）で、型コードは Python の版にも依存します。np.array(...) は型を推定し、dtype で明示もできます。typecode とは別の指定方式です。

同じ演算子でも意味が変わる

```
a_std = ar.array("d", [1.0, 2.0])
a_np = np.array([1.0, 2.0])
print((a_std * 2).tolist())
print(a_np * 2)
```

出力

```
[1.0, 2.0, 1.0, 2.0]
[2. 4.]
```

同じ型コードの標準 array の+は連結、* 2 は繰り返し。**は TypeError です。
ndarray の+、*、**は要素ごとの演算、@は行列積です。

標準 array の追加・削除

すべて元配列を書き換えます。

append(x)・extend(xs)：末尾へ追加。
insert(i, x)：添字位置へ追加。
del a[i]：その要素を削除。
pop(i)：削除して値を返す。i 省略時は末尾。
remove(x)：最初に一致する値を 1 つ削除。

```
a = ar.array("d",
    [1, 2, 2, 3])
a.append(4)
print(a.pop())
a.remove(2)
print(a.tolist())
```

出力

```
4.0
[1.0, 2.0, 3.0]
```

範囲外・空の pop は IndexError、見つからない値への remove は ValueError です。

NumPyの追加は返り値を受け取る

ndarrayにappendメソッドはありません。np.append・np.insertは新配列を返します。

```
a_np = np.array([1.0, 2.0, 2.0, 3.0])
grown_np = np.append(a_np, 4)
inserted_np = np.insert(a_np, 1, 9)
print(a_np, grown_np, inserted_np,
   sep="\n")
```

出力

```
[1. 2. 2. 3.]
[1. 2. 2. 3. 4.]
[1. 9. 2. 2. 3.]
```

元に反映するには返り値を代入します。

NumPyの削除：値か、添字か

np.deleteは添字を指定し、条件式は条件に合う値を選びます。

```
a_np = np.array([1.0, 2.0, 2.0, 3.0])
print(np.delete(a_np, 1))
print(a_np[a_np != 2])
print(a_np)
```

出力

```
    [1. 2. 3.]
    [1. 3.]
    [1. 2. 2. 3.]
```

np.delete(a_np, 1)は添字１を削除。a_np[a_np != 2]は値２をすべて除きます。どちらも新配列を返します。直接の添字代入は元を更新します。

多次元配列の追加と axis

```
m = np.array([[1, 0], [0, 2]])
r = np.array([[-1, 0]])
joined = np.concatenate((m, r), axis=0)
flat = np.append(m, r)
print(joined.shape, flat.shape)
```

出力

```
(3, 2) (6,)
```

axis=0 は行、axis=1 は列。結合する軸以外の大きさをそろえます。

np.append・np.insert・np.delete は既定の axis=None で平坦化。np.concatenate は既定の axis=0 です。

スライスはコピーか、共有か

```
a_std = ar.array("d", [1, 2, 3, 4])
s_std = a_std[1:3]
s_std[0] = 99
a_np = np.array([1., 2., 3., 4.])
s_np = a_np[1:3]
s_np[0] = 99
print(a_std.tolist(), a_np, sep="\n")
```

出力

```
[1.0, 2.0, 3.0, 4.0]
[ 1. 99.  3.  4.]
```

view と copy・高度な添字

```
a_np = np.array([1., 2., 3., 4.])
selected_np = a_np[[1, 2]]
selected_np[0] = 99
copy_np = a_np[1:3].copy()
copy_np[0] = -1
print(a_np)
```

出力

```
[1. 2. 3. 4.]
```

標準 array のスライスはコピー、NumPy の基本スライスは view です。整数配列・条件配列による選択結果と .copy() も独立配列です。直接の添字代入は元を更新し、b = a は同じオブジェクトを参照します。

shape・reshape・転置・行列積

```
matrix_np = np.arange(4).reshape(2, 2)
vector_np = np.array([1, 2])
print((matrix_np @ vector_np).tolist())
print(matrix_np.T.tolist())
print(vector_np.shape,
   vector_np.T.shape)
```

出力

```
    [2, 8]
    [[0, 2], [1, 3]]
    (2,) (2,)
```

reshape は要素数を保ち、結果は view または copy です。@は行列積。.T は２次元では転置、１次元では形を変えません。

整数型への小数代入：標準 array

array('i') の要素は C の int 型です。小数はそのまま代入できません。

```
int_std = ar.array("i", [1, 2])
int_std[0] = -2.5
```

出力

```
TypeError
```

整数ndarrayへ小数を代入すると

```
int_np = np.array([1, 2], dtype=int)
int_np[0] = -2.5
float_np = np.array([1, 2],
                    dtype=float)
float_np[0] = -2.5
print(int_np, float_np)
```

出力

```
[-2  2] [-2.5  2. ]
```

この代入では小数部が0方向に落ち、-2になります。小数を保持するならdtype=floatで作ります。これはこの要素代入例の結果です。

相互変換とコピー

```
source_std = ar.array("d", [0, 1])
converted_np = np.array(source_std)
converted_np[0] = 99
print(source_std.tolist(),
   converted_np)
print(np.sin(source_std),
   type(np.sin(source_std)))
```

出力

```
    [0.0, 1.0] [99. 1.]
    [0. 0.84147098] <class 'numpy.ndarray'>
```

np.sin は標準 array を受け取り ndarray を返します。共有・コピー条件は変換方法で異なります。

2次元配列を標準arrayへ戻す

1次元なら ar.array('d', nd.tolist()) で戻せます。2次元の tolist() は入れ子なので、平坦化します。

```
matrix_np = np.array([[1.0, 2.0],
   [3.0, 4.0]])
flat_std = ar.array("d",
   matrix_np.flatten().tolist())
print(flat_std.tolist())
```

出力

```
    [1.0, 2.0, 3.0, 4.0]
```

flatten() 後は値が並ぶだけで、(2, 2) の形は残りません。形が必要なら ndarray で扱います。

自由落下を標準arrayで計算する

静止・原点から落下、下向きを正、空気抵抗なし、$g = 10.0\ \mathrm{m/s^2}$ とします。$x(t) = \frac{1}{2}gt^2 = 5.0t^2 \quad (t \geq 0)$

```
times_std = ar.array("d", [0., 1., 2.])
positions_std = ar.array("d")
for time_value in times_std:
    positions_std.append(
        5.0 * time_value ** 2)
print(positions_std.tolist())
```

出力

```
[0.0, 5.0, 20.0]
```

時刻を順番に走査し、各時刻の位置を append で追加します。

同じ自由落下を ndarray で計算する

同じ条件・時刻では、配列演算で全位置をまとめて求められます。

```
times_np = np.array([0., 1., 2.])
positions_np = 5.0 * times_np ** 2
print(positions_np)
```

出力

```
[ 0.  5. 20.]
```

1 次元の同型値を標準ライブラリだけで扱うなら array.array、一括計算・多次元・行列計算なら ndarray が便利です。速度は実際の条件で測ります。

仕様・使用例：Python：array、NumPy：基本操作、コピーと view

np.append、np.delete、np.array